

HOM Studio OS v2 - Code-First UI Strategy Addendum

Generated: 2026-05-25

This addendum updates the previous PRD and implementation pack. It changes the frontend strategy from Figma-first to code-first, while still preserving user journey discipline, design quality, and production safety.

Bottom Line

It is acceptable to let Codex build the frontend directly from the current website/screenshots if the project uses a strict code-first design system, reusable components, Storybook/demo states, and visual quality gates. It is not acceptable to let Codex freely generate disconnected pages without design tokens, route-level specs, or user journey maps.

Research References Used for This Code-First UI Update

These sources were used to adjust the strategy from Figma-first to code-first UI development. Re-check current docs before implementation because tool capabilities and pricing can change.

- Nielsen Norman Group - Journey Mapping 101: <https://www.nngroup.com/articles/journey-mapping-101/>
- Nielsen Norman Group - User Journeys vs. User Flows: <https://www.nngroup.com/articles/user-journeys-vs-user-flows/>
- Nielsen Norman Group - Dashboards: Making Charts and Graphs Easier to Understand: <https://www.nngroup.com/articles/dashboards-preattentive/>
- Nielsen Norman Group - Data Visualizations for Dashboards: <https://www.nngroup.com/videos/data-visualizations-dashboards/>
- Baymard - Accounts & Self-Service UX examples and research catalog: <https://baymard.com/research/self-service>
- Storybook official docs: <https://storybook.js.org/docs>
- shadcn/ui official docs: <https://ui.shadcn.com/docs>
- Motion.dev official docs: <https://motion.dev/docs/react>
- 21st.dev Agent Elements repository: <https://github.com/21st-dev/agent-elements>
- OpenAI function calling and structured outputs docs: <https://developers.openai.com/api/docs/guides/function-calling>

15 - Code-First UI Strategy Without Figma

Purpose

This document updates the project strategy. The project no longer assumes that Figma must exist before frontend development starts.

Instead, the frontend will be designed and built directly in code, using the current HOM dashboard screenshots as the visual baseline, then improving the user journey, information architecture, visual hierarchy, and interaction quality.

Verdict

This strategy is acceptable and practical for a solo developer using Codex, but only if the frontend is built with a strict component system, Storybook, visual regression tests, and clear screen specifications.

Without Figma, the source of truth becomes:

1. `/docs UI specifications`
2. `design tokens in code`
3. `reusable components`
4. `Storybook stories`
5. `screenshot regression tests`
6. `implemented pages`

Do not treat AI-generated frontend code as the source of truth by itself.

Why This Is Reasonable

A Figma-first workflow is useful for teams, stakeholders, and design handoff. But this project is being executed by a solo developer and the current product already has visible UI references from screenshots. A code-first approach can be faster, cheaper, and easier to iterate if the design system is enforced in the repository.

The risk is that the app becomes visually inconsistent. To avoid that, build the design system before building all pages.

Current HOM Visual Baseline

The current interface has these recognizable visual traits:

- `Dark left sidebar`
- `Minimal off-white dashboard background`
- `Warm beige/gold accent colors`
- `Premium clinical studio branding`
- `Large executive dashboard title treatment`
- `Card-based analytics sections`

- Wide tables for finance and LTV
- Tab navigation across executive modules
- WhatsApp/live-chat operational view
- Human intervention toggle for AI chat
- Approval cards for reimbursements, notes, and blasts

The new version should preserve the brand mood, but improve clarity, hierarchy, consistency, responsiveness, accessibility, and error handling.

New Design Direction

The new UI should feel like:

Premium clinical operations dashboard
 not generic SaaS template
 not over-animated startup landing page
 not consumer fitness app
 not medical EMR clutter

Target keywords:

calm
 precise
 premium
 clinical-safe
 executive
 operational
 trustworthy
 human-controlled AI

Code-First Frontend Rules

Rule 1 - Build components first

Before building pages, create these components:

AppShell
 SidebarNavigation
 TopSearchBar
 PageHeader
 TabNavigation
 MetricCard
 InsightCard
 StatusBadge
 DataTable
 EmptyState
 ErrorState
 LoadingSkeleton
 ApprovalCard

ChatBubble
HumanInterventionToggle
FileUploadPanel
KnowledgeSourceCard
ProcessingTimeline
ModelPicker
ToolCallCard

Rule 2 - Every page must have four states

Each screen must include:

loading state
empty state
error state
success/data state

The previous screenshots often show indefinite Loading... The rebuilt system must avoid that.

Rule 3 - Use mock data first, API later

Codex should build pages in this order:

1. Static mock layout
2. Storybook story
3. Responsive refinement
4. Accessibility check
5. API contract integration
6. Real data integration
7. Playwright visual snapshot

Do not connect API before the UI state model is stable.

Rule 4 - UI is not allowed to contain business logic

Frontend components may render state, but business rules must live in back-end/domain modules.

Bad:

Frontend calculates whether a practitioner can be double-booked.

Good:

Frontend calls appointment availability endpoint and displays returned allowed actions.

Rule 5 - Motion must be restrained

Use Motion.dev for:

modal transitions
sidebar collapse
chat message arrival
card entrance
file ingestion timeline
approval card movement
loading transitions

Do not use heavy animated backgrounds, exaggerated bounce, or scroll effects on financial/clinical screens.

Rule 6 - AI UI must expose sources and actions

Every AI-generated business answer should display:

source data period
knowledge version
tool calls used
confidence/risk label
whether the result is read-only or action-required

This is especially important for finance, clinical, appointment, and WhatsApp decisions.

Recommended Frontend Stack

Next.js
React 19
TypeScript strict
Tailwind CSS
shadcn/ui
Radix UI
Motion.dev
Storybook
Playwright
21st Agent Elements for AI/chat interfaces where useful

Figma Later, Not Now

Figma can be added later if:

- the owner wants a brand redesign
- another designer joins
- UIPro/Figma-to-code becomes useful
- marketing website needs more visual experimentation

For now, do not block development because Figma is missing.

Definition of Done for Code-First UI

A screen is not done until:

- it has loading, empty, error, and data states
 - it is responsive at desktop/tablet widths
 - it uses design tokens, not random colors
 - it is represented in Storybook when reusable
 - it passes TypeScript checks
 - it passes basic Playwright navigation test
 - it has no hardcoded production data
 - it has no secret keys
 - it matches HOM visual direction
-

16 - User Journey Maps and Information Architecture

Purpose

This document defines the user journeys before Codex builds the screens. A code-first build is acceptable only if the user journey is explicit.

Primary Users

Studio Director / Business Owner
Front Desk / Admin
Practitioner
Finance Admin
Marketing / AI Operator
AI Service Account

Journey 1 - Studio Director Daily Command

User goal

The Studio Director wants to know what requires attention today: revenue, appointments, practitioner status, unresolved chat, pending approvals, and AI insights.

Entry point

/dashboard/executive-command

Flow

Open dashboard

- > See today's risk and opportunity summary
- > Review revenue/client/utilization KPIs
- > Review pending approvals
- > Review AI insights
- > Drill down into financial, LTV, appointment, or chat modules
- > Approve, reject, or delegate action

UI implications

- Dashboard should not show every metric at once.
- Top section should answer: What needs attention now?
- Use alert/risk cards before vanity metrics.
- Every KPI should have period label and source status.
- AI insight must show whether it is based on live data or cached summary.

Success criteria

Director can understand studio health within 60 seconds.
Director can reach any critical pending approval within 2 clicks.
Director is not forced to ask AI just to see basic KPIs.

Journey 2 - Front Desk Appointment Operation

User goal

Admin wants to create, reschedule, cancel, and check appointment status quickly while avoiding double booking and wrong practitioner assignment.

Entry point

/appointments

Flow

Search client

- > Open client context
- > Create or reschedule appointment
- > Select service, practitioner, date, time
- > Backend checks availability and conflicts
- > Admin confirms
- > System sends WhatsApp confirmation or queues message
- > Appointment history is updated

UI implications

- Appointment screen should be operational, not decorative.
- Calendar and list view must both exist.
- Reschedule flow must show previous and new schedule.
- System must clearly show whether WhatsApp confirmation was sent, queued, or failed.
- If schedule is loading too long, show retry and fallback list view.

Success criteria

Admin can create appointment in under 90 seconds.
Admin can reschedule without losing history.
Admin sees conflict explanation, not just generic error.

Journey 3 - Practitioner Clinical Session

User goal

Practitioner wants to see today's sessions, understand client context, write session notes, and lock notes after completion.

Entry point

/practitioner/today or /sessions/:id

Flow

Open daily schedule
-> Select next client
-> Review condition summary and recent notes
-> Conduct session
-> Write session note
-> Mark appointment done
-> Lock note
-> Suggest next session or follow-up

UI implications

- Clinical condition data must be concise and permission-controlled.
- Do not overload practitioner with finance details.
- Session note editor needs autosave and locked state.
- Unlock request should require reason.

Success criteria

Practitioner can understand client context within 30 seconds.
Finalized notes cannot be silently edited.
Sensitive case views are audit logged.

Journey 4 - Finance Monthly Close

User goal

Finance/Admin wants to reconcile revenue, expenses, commission, reimbursement, and export a report.

Entry point

/financials

Flow

Choose month/year

- > Review monthly summary cards
- > Review ledger rows
- > Add/edit operational expenses
- > Review practitioner commissions
- > Reconcile inconsistencies
- > Generate PDF report
- > Export and archive report

UI implications

- Summary cards and ledger must use same period filter.
- If data is missing, show missing-data status, not Rp 0 as fake truth.
- Every financial row must show category and source.
- Report generation should be async when slow.

Success criteria

Finance view never shows inconsistent period data.

Director can identify why a number changed.

PDF report is reproducible from stored data snapshot.

Journey 5 - Owner Updates AI Knowledge

User goal

Business owner wants to upload new pricing/SOP/FAQ/PDF/XLSX and adjust how the AI answers without editing code.

Entry point

/knowledge-studio

Flow

Upload file

- > System extracts text/table/image information
- > Owner reviews extracted knowledge
- > Owner assigns scope
- > Owner runs test questions
- > System reports pass/fail, source usage, risk
- > Owner publishes or rejects
- > Previous version remains rollbackable

UI implications

- Never auto-publish uploaded knowledge.
- Make extraction review easy for non-technical users.
- For spreadsheets, show sheet/column mapping.
- Show affected agents before publishing.

Success criteria

Owner can update FAQ/pricing without developer help.
AI answer sources are visible.
Knowledge update can be rolled back.

Journey 6 - Behavior Learning From Chat

User goal

Owner wants to understand customer behavior from WhatsApp conversations without manually reading every chat.

Entry point

/behavior-intelligence

Flow

System processes conversations

- > Extracts intent, topic, objection, sentiment, schedule preference
- > Aggregates weekly insights
- > Detects unanswered questions
- > Suggests knowledge updates or campaign segments
- > Owner approves changes
- > Knowledge Studio publishes update if needed

UI implications

- Do not show raw private chats as the first view.
- Show aggregate insight first.
- Provide sample conversations only when user has permission.
- Every AI-generated insight should have evidence count.

Success criteria

Owner can see top customer questions each week.
Owner can detect FAQ gaps.
Owner approves improvements before AI behavior changes.

Information Architecture

Primary navigation:

Overview
Appointments
Clients
Live Chat
Knowledge Studio
Behavior Intelligence
Financials
AI Business Agent
Approvals
Settings

Executive Command tabs:

Strategic Overview
Chronic Case Registry
Team Attendance
Financial Strategy
Client LTV & Milestones
User Management
Approvals & Payroll

Recommended change:

Keep the Executive Command tabs, but do not hide operational modules only inside it. Appointment, Chat, Clients, and Knowledge Studio should be direct sidebar items.

17 - Code-First Screen Specifications

Purpose

This document turns the current HOM screenshots into actionable screen specifications for Codex. Each screen should be built first with mock data, then connected to API.

Global Layout

App Shell

Required elements:

- dark left sidebar
- HOM logo area
- current user card
- sidebar navigation
- top search bar
- help icon
- notification bell
- user menu
- main content area

Visual behavior

- Sidebar can collapse.
- Active nav item uses beige/gold accent.
- Main area uses warm off-white background.
- Cards use white or very light neutral surface.
- Use subtle borders and shadow; avoid heavy gradients.

Screen 1 - Executive Command / Strategic Overview

Route:

`/dashboard/executive-command`

Top sections:

1. Page header
2. Critical attention strip
3. KPI cards
4. Revenue growth chart
5. Practitioner metrics
6. AI intelligence panel
7. Studio utilization
8. Token/cost monitor

KPI cards:

Monthly Revenue
Active Clients
Performance Bonus Pool
AI Lead Conversion
Pending Approvals
Open Chat Interventions

Required states:

loading: skeleton KPI cards and chart placeholder
empty: no dashboard data for selected period
error: dashboard data failed to load with retry
success: data with period/source labels

Important UX change:

The current dashboard shows many metrics. The new dashboard should start with Attention Required cards:

- 3 pending WhatsApp blast approvals
- 2 failed reminders
- 1 finance reconciliation issue
- 4 appointments need reschedule confirmation

Screen 2 - Appointments

Route:

/appointments

Required modules:

- appointment status filters
- calendar/list toggle
- date navigation
- new appointment button
- search client/session
- appointment cards/table
- conflict warning modal
- reschedule side panel

Statuses:

scheduled
confirmed
reschedule_requested
done
cancelled
no_show

Important UX change:

Do not show endless Loading `schedule...` If loading exceeds timeout, show:

Schedule still loading

[Retry] [Open list view] [Report issue]

Screen 3 - Live Chat

Route:

`/live-chat`

Required modules:

- conversation list
- search conversations
- practitioner/client tags
- message thread
- AI draft bubble
- manual intervention toggle
- suggested replies
- action panel for reschedule/booking/FAQ
- WhatsApp blast entry point

Safety requirements:

- AI cannot directly reschedule without backend availability check.
- Human takeover stops auto-reply for that conversation.
- Sensitive clinical messages get escalation label.

Screen 4 - Knowledge Studio

Route:

`/knowledge-studio`

Required modules:

- upload panel
- knowledge source list
- processing timeline
- extraction review
- scope selector
- spreadsheet mapping mode
- test lab
- publish/rollback controls

Knowledge source statuses:

uploaded

processing

review_required

tested
published
failed
archived

Screen 5 - Behavior Intelligence

Route:

/behavior-intelligence

Required modules:

- top intents this week
- unanswered questions
- lead objections
- reschedule reasons
- practitioner demand mentions
- customer segments
- suggested knowledge updates

Important UX rule:

Show aggregate trend first. Raw chat examples should only appear after clicking into an insight and only for authorized roles.

Screen 6 - Financial Strategy

Route:

/financials

Required modules:

- year/month filter
- summary cards
- cash flow chart
- therapist commission panel
- studio ledger statement
- operational expense input
- reconciliation warnings
- PDF report export

Important UX change:

If summary and ledger disagree, show reconciliation warning. Never silently show Rp 0 if ledger contains data.

Screen 7 - Approvals & Payroll

Route:

/approvals

Required modules:

- pending reimbursement claims
- clinical note unlock requests
- WhatsApp blast clearances
- processed request history
- payroll integration logs

Approval card actions:

approve
reject
request more info
open evidence
view audit trail

Screen 8 - Chronic Case Registry

Route:

/clinical-cases

Required modules:

- active case cards
- condition label
- assigned practitioner
- session count
- status
- restricted detail view
- audit log on open

Safety requirement:

Clinical case details must not be sent to AI unless masked/summarized and explicitly allowed by policy.

18 - Design System Without Figma

Purpose

This document defines how to keep UI consistent when there is no Figma file.

Source of Truth

The source of truth is code:

Tailwind tokens
CSS variables
component props
Storybook stories
Playwright screenshots
Motion presets

Design Tokens

Create tokens for:

color
spacing
radius
shadow
typography
motion
z-index
status colors
chart colors

Color System

Suggested semantic tokens:

--background-app
--background-sidebar
--background-card
--foreground-primary
--foreground-muted
--accent-gold
--accent-gold-muted
--border-subtle
--status-success
--status-warning
--status-danger
--status-info

Do not hardcode colors inside feature components.

Typography

Define:

page-title
section-title
card-title
metric-value

metric-label
body
caption
table-header
chat-message

Component Naming

Use this structure:

components/ui/	generic shadcn-based primitives
components/hom/	HOM-specific reusable components
components/agent/	AI/chat/knowledge components
features/	page-specific feature modules

Storybook Requirements

Each reusable component should have stories for:

default
loading
empty
error
long text
mobile/narrow container
permission denied where relevant

Storybook is important because, without Figma, it becomes the visible catalog of the design system.

Motion Guidelines

Create motion presets:

fadeInSubtle
slideUpCard
sidebarCollapse
messageIn
modalScale
processingStep

Every motion must support reduced motion preferences.

Data Visualization Guidelines

Dashboard charts should emphasize clarity over decoration.

Use:

line chart for trends
bar chart for comparisons
table for exact financial numbers
status badges for operational state
attention cards for action required

Avoid:

3D charts
too many colors
unlabeled axes
charts without period labels
mixing revenue and client count without clear axes

Accessibility Checklist

keyboard navigation works
visible focus state
aria labels for icon buttons
sufficient color contrast
forms have labels
errors are explained in text
motion can be reduced
chat bubbles readable by screen readers

Editable Later

Because the UI is code-first, future edits can be made by changing:

design tokens
component variants
Storybook examples
page configuration
copy/content in database
business rules in Knowledge Studio

This makes the system editable without needing a complete Figma rebuild.

19 - Code-First UI Codex Prompts

Purpose

Use these prompts after the foundation docs are in the repo. Do not run them all at once. Paste one prompt at a time into Codex.

Prompt UI-01 - Create Design Tokens and App Shell

You are working on HOM Studio OS v2.

Task:

Create the code-first design foundation and app shell.

Read:

- docs/15_CODE_FIRST_UI_STRATEGY.md
- docs/17_SCREEN_SPECIFICATIONS.md
- docs/18_DESIGN_SYSTEM_WITHOUT_FIGMA.md

Implement:

1. Tailwind/CSS variables for HOM visual tokens.
2. AppShell with dark sidebar, top search, notification area, user menu placeholder, and main content area.
3. SidebarNavigation with active state.
4. PageHeader component.
5. LoadingSkeleton, EmptyState, ErrorState components.
6. Storybook stories if Storybook already exists; otherwise create component examples in a new storybook.

Rules:

- Do not connect real APIs yet.
- Use mock data only.
- Do not add paid services.
- Keep commits small.
- Ensure TypeScript passes.

Prompt UI-02 - Build Executive Command Mock Page

Task:

Build the Executive Command strategic overview page using mock data.

Read:

- docs/16_USER_JOURNEY_MAPS.md
- docs/17_SCREEN_SPECIFICATIONS.md
- docs/18_DESIGN_SYSTEM_WITHOUT_FIGMA.md

Implement:

1. Route /dashboard/executive-command.
2. Attention Required section.
3. KPI cards.
4. Revenue growth panel.
5. Practitioner metrics panel.
6. Studio utilization card.
7. AI cost/token monitor card.
8. Loading, empty, error, and success states.

Rules:

- Keep the visual style close to current HOM: premium, calm, off-white, dark sidebar, gold a
- Do not over-animate.
- Do not connect database yet.

Prompt UI-03 - Build Appointments Mock Page

Task:

Build the Appointments page using mock data and strong UX states.

Implement:

1. Route /appointments.
2. Status filters: All, Scheduled, Done, Reschedule, Cancelled, No-show.
3. Calendar/List toggle.
4. Date navigation.
5. New Appointment button.
6. Appointment cards/table.
7. Reschedule side panel mock.
8. Conflict warning mock.
9. Loading timeout fallback UI.

Rules:

- No backend integration yet.
- Focus on workflow clarity.
- Show clear empty and error states.

Prompt UI-04 - Build Live Chat and AI Handoff Mock Page

Task:

Build the Live Chat operations page.

Implement:

1. Route /live-chat.
2. Conversation list with tags.
3. Message thread.
4. AI draft response bubble.
5. Manual intervention toggle.
6. Suggested replies panel.
7. Action panel for reschedule request.
8. Safety labels for sensitive clinical/refund messages.

Rules:

- AI must appear as draft/human-controlled in mock.
- Do not send real WhatsApp messages.
- Use 21st Agent Elements only if it fits the stack and does not add unstable dependencies.

Prompt UI-05 - Build Knowledge Studio Mock Page

Task:

Build the Knowledge Studio UI for owner-adjustable AI knowledge.

Implement:

1. Route /knowledge-studio.
2. File upload panel for PDF, DOCX, XLSX, CSV, image.
3. Knowledge source list.
4. Processing timeline.
5. Extraction review panel.
6. Scope selector.
7. Spreadsheet mapping mock.
8. Test Lab mock.
9. Publish/Rollback controls.

Rules:

- No actual file parsing yet.
- UI must make it obvious that uploaded knowledge is not active until published.

Prompt UI-06 - Build Behavior Intelligence Mock Page

Task:

Build the Behavior Intelligence dashboard.

Implement:

1. Route /behavior-intelligence.
2. Top intents chart/list.
3. Unanswered questions list.
4. Lead objections.
5. Reschedule reason summary.
6. Practitioner demand mentions.
7. Suggested knowledge updates.
8. Permission-safe sample conversation drawer.

Rules:

- Use aggregate insights first.
- Do not expose raw chat by default.

Prompt UI-07 - Add Motion Pass

Task:

Add restrained Motion.dev animations to completed mock screens.

Implement:

1. Card entrance animation.

2. Sidebar collapse transition.
3. Chat message arrival animation.
4. Modal/side-panel transition.
5. File processing timeline animation.
6. Reduced-motion support.

Rules:

- Keep finance/clinical screens calm.
- No parallax or distracting effects.
- Do not animate critical numbers in a way that reduces readability.

Prompt UI-08 - Add Visual Quality Gates

Task:

Add frontend quality checks.

Implement:

1. Playwright smoke test for main routes.
2. Basic screenshot tests for key pages if project setup supports it.
3. Accessibility checks for navigation, buttons, and forms.
4. Lighthouse CI placeholder or script if appropriate.
5. README instructions for running UI checks.

Rules:

- Keep setup simple.
- Do not block development with overly complex CI.